Does Orchestration Earn Its Cost?

A Comparative Study of Single-Agent vs. Multi-Agent LLM Systems on Software Engineering Tasks

CS 540 Final Project Report

Niharika Belavadi Shekar, Moulshree Guleria, Karthik Ragi, Gautham Satyanarayana

May 2026

Contents

1 Introduction

1.1 Original Goals of the Project

The original goal of this project was to compare *single-agent* and *multi-agent* large language model (LLM) systems on two canonical software engineering tasks: requirements engineering (RE) classification and code generation. For requirements engineering, the system would receive a natural-language requirement sentence and classify it as a Functional Requirement (FR), Non-Functional Requirement (NFR), or Not a Requirement (NONE), with an NFR subtype if applicable. For code generation, the system would receive a programming problem and generate correct Python code, evaluated by automated test execution.

The central research question was whether multi-agent orchestration, decomposing a task across a Planner, Extractor, Critic, and Coder, improves output quality over a simpler single-shot approach, and whether any quality gain justifies the additional computational cost.

1.2 Modified Goals

The RE classification task was redesigned partway through the project into *requirements elicitation*. Rather than classifying pre-existing sentences, the system now receives a natural-language use-case description and generates a complete list of requirements from scratch. This modification was made for two reasons:

1. Classification of individual sentences is a low-complexity task where modern LLMs achieve near-ceiling performance in a single call, making it a poor testbed for comparing single-agent vs. multi-agent systems.
2. Elicitation better reflects the actual challenge in requirements engineering practice: generating requirements from an informal description, not labelling pre-written ones.

To support this redesign, the system input was redesigned from a bare use-case sentence to a structured **Business Requirements Document (BRD)**. A BRD is the artefact a requirements engineer typically receives in practice: a multi-section business document covering problem statement, business objectives and success criteria, stakeholders, current-vs-future state, business rules, scope, constraints, and assumptions. Using a BRD as input makes the elicitation task significantly more realistic and tests whether orchestration adds value when the context is rich and structured, as it would be on a real engagement.

BRDs were synthetically generated from ground-truth requirement lists using the Claude API (`scripts/prepare_re_elicitation.py`). Claude was prompted as a senior business analyst to *reverse-engineer* a realistic BRD from each project's known requirements, writing in business prose across all eight required sections without reproducing the requirements verbatim. This gave a controlled evaluation pair: the BRD is the system input; the original ground-truth requirements are the evaluation target. The datasets changed from NICE (classification) and SecReq to NICE (elicitation) and PURE (elicitation).

A second modification was the addition of **BigCodeBench-Hard** as a third code generation benchmark. HumanEval+ and MBPP+ both produced 100% pass@1 for single-agent and V1—too easy to reveal differences between systems. BigCodeBench-Hard was added to stress-test the repair loop on real-world library-dependent tasks.

1.3 Assumptions

- **Platform**: macOS (Darwin 25.0.0, Apple Silicon).
- **Programming language**: Python 3.14.

- **LLM**: Anthropic `claude-sonnet-4-6`, temperature = 0 for all systems and all runs.
- **Orchestration framework**: LangGraph (LangChain) for multi-agent state graphs.
- **Evaluation**: Code correctness assessed by subprocess execution (isolated, 10–30 second timeout). RE quality assessed by semantic similarity, not exact match.
- **Cost model**: Tokens estimated as (input characters + output characters) / 4, consistent across all systems.
- **Reproducibility**: `temperature=0` throughout; `RANDOM_SEED=42` for all dataset splits.
- **Single model assumption**: All agents in the multi-agent systems use the same underlying model. No agent has access to external tools, search, or memory beyond the current task's state.

1.4 Definitions

pass@1 Fraction of code generation problems where the generated code passes all test cases on the first attempt.

Semantic F1 Harmonic mean of semantic coverage (recall) and semantic precision, computed via cosine similarity of sentence embeddings with threshold 0.60.

Single-agent One LLM call per task, with up to 2 retries on compile/parse failure.

Multi-agent V1 LangGraph pipeline: Planner → Extractor → Critic → Coder (only for Code generation) → Test Runner, with a repair loop.

Multi-agent V2 V1 extended with either a Test Critic node (code generation) or a domain SME advisory node (requirements elicitation).

Repair loop Coder → Test Runner → Coder cycle triggered when tests fail, up to `MAX_REPAIR_ITERATIONS=3`.

Budget guard Per-task hard limits (`MAX_LLM_CALLS=10`) for both tasks, and (`MAX_TOKENS=8000`) for code generation, (`MAX_TOKENS=30000`) for requirements elicitation enforced at every routing function to prevent runaway loops.

Context trace A per-invocation log appended to graph state recording which context components (plan, critique, test error) were present at each coder call.

1.5 Summary of Approach

We built three systems on the same underlying model and evaluated them on five benchmarks under identical conditions. For code generation: HumanEval+ (164 problems), MBPP+ (378 problems), and BigCodeBench-Hard (148 problems). For requirements elicitation: NICE (15 projects) and PURE (15 documents).

After observing that all systems achieved 100% pass@1 on HumanEval+ and MBPP+, we added BigCodeBench-Hard to expose regime differences. During analysis of BigCodeBench failures, we identified two context-loss bugs in the V1 repair loop, the planner's output was silently dropped from repair prompts, and unittest-style tests were never executed inside the graph. We fixed both bugs and re-evaluated.

1.6 Project Deliverables

1. **Three implemented systems**: single-agent, multi-agent V1, multi-agent V2, for both code generation and requirements elicitation tasks.
2. **Five benchmark evaluations**: HumanEval+, MBPP+, BigCodeBench-Hard, NICE, PURE.
3. **Context trace mechanism**: per-repair-attempt logging of context availability and prompt composition metrics.
4. **Bug analysis and fix**: documented identification and correction of two context-loss bugs in the V1 repair loop.
5. **Run scripts with incremental saving**: resume-safe evaluation scripts that write results per-task (no data loss on crash or credit exhaustion).
6. **Results reports**: six markdown reports covering all benchmarks and systems.

1.7 Contribution of Individual Team Members

All four team members collaborated during the initial phase to collectively design the architecture of all four systems, single-agent and multi-agent pipelines for both requirements elicitation and code generation and jointly researched the available datasets, aligned on evaluation metrics, and defined the project deliverables.

- **Karthik Ragi**: Implemented the single-agent requirements elicitation pipeline and ran evaluations on the NICE and PURE datasets.
- **Moulshree Guleria**: Implemented the multi-agent requirements elicitation graph including the Critic loop and SME advisory node, and ran evaluations on the NICE and PURE datasets.
- **Niharika Belavadi Shekar**: Implemented the single-agent code generation system and ran evaluations on the HumanEval+, MBPP+, and BigCodeBench-Hard benchmarks.
- **Gautham Satyanarayana**: Implemented the multi-agent code generation graph including the repair loop and context-trace mechanism, and ran evaluations on the HumanEval+, MBPP+, and BigCodeBench-Hard benchmarks.

2 Background Research

2.1 Primary Research

Software and frameworks evaluated: We evaluated several orchestration frameworks before settling on LangGraph:

- **LangGraph** (LangChain AI): Selected for its stateful `StateGraph` abstraction, built-in support for conditional routing, and native integration with LangChain's LLM clients. The `Annotated[int, operator.add]` reducer pattern for accumulating token counts across nodes was a key design enabler.
- **Anthropic Claude API**: Used directly for all LLM calls. The structured-output capability (`with_structured_output`) via Pydantic schemas eliminated the need for manual JSON parsing.
- **EvalPlus**: Evaluation library providing HumanEval+ and MBPP+ datasets with extended test suites. Loaded via `evalplus.data.get_human_eval_plus()` and `get_mbpp_plus()`.

- **BigCodeBench-Hard**: Loaded via HuggingFace `datasets` library (`bigcode/bigcodebench-hard`, split `v0.1.4`).
- **sentence-transformers**: Used for semantic evaluation of generated requirements via the `all-MiniLM-L6-v2` model.

Tooling and infrastructure: Incremental JSONL writing with per-task append (`append_jsonl`) and resume support (`load_completed_ids`) were developed after losing data to API credit exhaustion mid-run on MBPP+. A monitor-based background process pattern using `tail -f | grep` was used throughout for real-time progress tracking.

2.2 Secondary Research

Papers and documentation read.

- **EvalPlus** [1]: Liu et al. (2023). Introduced the augmented test suite philosophy and the pass@k metric. Used to understand why 100% pass@1 on HumanEval does not mean the code is truly correct.
- **BigCodeBench** [2]: Zhuo et al. (2024). Motivated the switch to a harder benchmark. The paper documents how real-world library usage (pandas, sklearn, Flask) dramatically reduces pass@1 for all state-of-the-art models.
- **NICE** [3]: Fantechi et al. (ICSE 2025). Source of the RE elicitation dataset. Read to understand the project-level grouping of requirements and the FR/NFR taxonomy.
- **PURE** [4]: Ferrari et al. (IEEE RE 2017). Source of the formal specification dataset. The XML schema parsing required understanding both Schema A (explicit `<req>` tags) and Schema B (leaf `<p>` nodes filtered by "shall"/"must").
- **Sentence-BERT** [5]: Reimers & Gurevych (2019). Motivated the use of cosine similarity over sentence embeddings rather than exact-match or BLEU-style metrics for requirement comparison.
- **LangGraph documentation**: The `StateGraph` API, reducer semantics, and `add_conditional_edges` routing. Critical for understanding how state is merged across nodes and why missing TypedDict keys cause silent data loss.

2.3 What Was Most Helpful

The BigCodeBench paper was the most directly influential. It changed the project's experimental design: without it, the study would have concluded (incorrectly) that multi-agent systems add no value over single-agent on any code generation task, because HumanEval+ and MBPP+ are too easy. BigCodeBench-Hard revealed that the repair loop does fire on harder problems, and that its effectiveness depends critically on context integrity.

The LangGraph TypedDict documentation was the second most important resource. Understanding that state keys not declared in the `TypedDict` schema are silently dropped was essential to diagnosing the context-loss bug where the planner's output disappeared from repair prompts.

3 Approach to Goals

3.1 Strategy

The strategy was controlled ablation: hold the model, prompts, and evaluation conditions constant, and vary only the system architecture. This isolates the effect of orchestration from

confounds like model choice or prompt quality.

Three systems were built:

1. **Single-agent baseline**: One LLM call per task. Retries on compile/parse failure only. No inter-agent communication.
2. **Multi-agent V1**: A LangGraph pipeline with four specialised roles (Planner, Extractor, Critic, Coder) and an execution-feedback repair loop.
3. **Multi-agent V2**: V1 extended with an additional verification or advisory node (Test Critic for code generation; SME advisory for requirements elicitation).

Evaluation proceeded in two phases:

- **Phase 1**: Run all three systems on HumanEval+, MBPP+, NICE, and PURE. Observe that easy benchmarks do not differentiate systems.
- **Phase 2**: Add BigCodeBench-Hard; identify and fix context-loss bugs; re-evaluate V1.

3.2 Motivation for Strategy

The controlled-ablation design was chosen because the alternative, comparing across models or frameworks, conflates too many variables. By fixing everything except architecture, any quality or cost difference is attributable to the orchestration structure itself.

The choice to evaluate on both a *generation* task (code) and a *elicitation* task (requirements) was deliberate. These tasks differ in a key structural property: code generation has a binary oracle (tests pass or fail), while requirements elicitation has a fuzzy, semantic ground truth. We hypothesised that the repair loop would add more value where feedback is unambiguous (code) than where it is subjective (requirements). Results confirmed this: the critic loop hurt RE performance on both datasets.

The inclusion of BigCodeBench-Hard was motivated by the observation that single-agent achieves 100% on simpler benchmarks, there is no failure for the repair mechanism to recover from. A benchmark with a realistic failure rate (∼38% pass@1) was needed to test the repair loop at all.

3.3 Division of Work Among Team Members

The division of work was organized by system: since there were four systems and four team members, each member took ownership of one system following the initial collaborative brainstorming phase. The detailed breakdown of individual contributions is described in Section 1.7.

4 Implementation

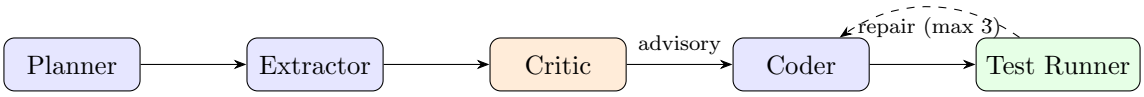
4.1 Software Design

The system is implemented as a Python package (`src/`) with the following module structure:

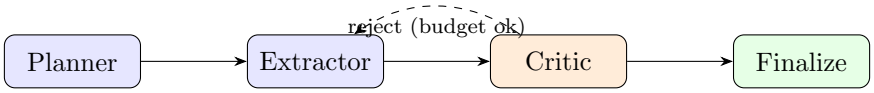
- `src/llm/client.py`: Central LLM client wrapping `ChatAnthropic`. Provides `get_structured_llm(schem` for Pydantic-typed outputs and `check_budget()` for per-task budget enforcement.
- `src/schemas/`: Pydantic v2 schemas for all structured outputs (`CodeSolution`, `REPrediction`, `GeneratedRequirement`) and LangGraph state TypedDicts (`CodeGenGraphState`, `REElicitationState`).
- `src/systems/single_agent/`: `CodeGenAgent` and `REElicitationAgent` , single-call implementations with retry logic.

- `src/systems/multi_agent/nodes/`: One file per node (`planner`, `extractor`, `critic`, `coder`, `test_runner`, `re_elicitation_*`, `re_sme_node`).
- `src/systems/multi_agent/codegen_graph.py`, `re_elicitation_graph.py`, etc.: LangGraph `StateGraph` wiring.
- `src/evaluation/`: `codegen_metrics.py` (pass@1, subprocess execution), `re_elicitation_metrics.py` (semantic F1 via sentence-transformers), `cost_tracker.py`.
- `scripts/`: Run scripts for each benchmark with incremental save and resume support.

Multi-agent code generation graph.



Multi-agent requirements elicitation graph.



Context loss fix. During analysis of BigCodeBench-Hard failures, two bugs were found in the V1 repair loop:

1. **Plan dropped on repair**: The Coder's repair prompt omitted the Planner's output (averaging 417 tokens of strategic context). Fixed by explicitly including `plan` and `critique` in all repair prompts.
2. **Unittest tests not executed**: `test_runner_node` concatenated solution and test code without injecting `unittest.main()`, so unittest-style test classes were defined but never run. Fixed by detecting unittest-style tests and injecting a runner.

A `context_trace` field was added to `CodeGenGraphState` (accumulated via `operator.add` reducer) to log context availability at every Coder invocation for future analysis.

4.2 External Components

Component	Version	Purpose
`langchain-anthropic`	latest	LLM client wrapping Claude API
`langgraph`	latest	Multi-agent state graph orchestration
`anthropic`	latest	Direct API access, structured outputs
`pydantic`	v2	Schema validation for all LLM outputs
`evalplus`	latest	HumanEval+ and MBPP+ datasets + evaluation
`datasets` (HuggingFace)	latest	BigCodeBench-Hard loading
`sentence-transformers`	$\geq$3.0	Semantic similarity for RE evaluation
`scikit-learn`	latest	F1/precision/recall computation
`python-dotenv`	latest	Environment variable management

4.3 Design of Empirical Studies

Requirements elicitation evaluation: Ground-truth requirements and generated requirements are embedded with `all-MiniLM-L6-v2`. A bipartite matching at cosine threshold 0.60 computes precision (generated $\rightarrow$ ground truth) and coverage/recall (ground truth $\rightarrow$ generated). Semantic F1 is the harmonic mean.

Code generation evaluation: Each problem's generated code is written to a temporary file along with the benchmark's test suite. A subprocess executes the file with a timeout (10s for EvalPlus, 30s for BigCodeBench-Hard). Exit code 0 = pass. For BigCodeBench, all third-party libraries (seaborn, flask-login, nltk, opencv-python-headless, etc.) are installed prior to evaluation using an import-name → pip-name mapping.

Cost tracking: A `CostTracker` object accumulates LLM calls and token estimates per task and writes a JSON summary at run end. Token estimates use the character-based approximation (chars)/4.

5 Results

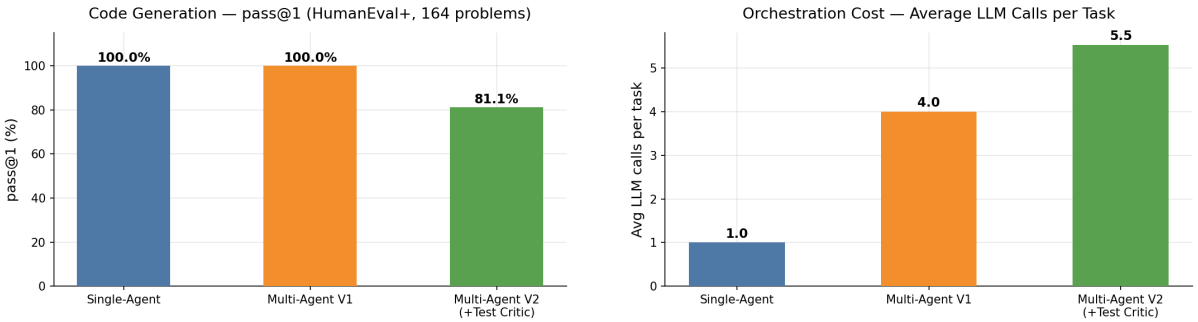
5.1 Empirical Results

5.1.1 Code Generation

Table 1: Code generation results: all systems and benchmarks.

System	Dataset	N	pass@1	Avg Calls	Avg Tokens
Single-Agent	HumanEval+	164	100.0%	1.0	263
	MBPP+	378	100.0%	1.0	138
	BigCodeBench-Hard	148	38.5%	1.0	836
Multi-Agent V1	HumanEval+	164	100.0%	4.00	2,037
	MBPP+	378	100.0%	4.00	1,531
	BigCodeBench-Hard	148	37.8%†	4.49	5,718
Multi-Agent V2	HumanEval+	164	81.1%	5.54	3,024
	MBPP+	357*	82.6%	5.80	∼2,219

† Pre-bugfix. * 357/378 completed.



(a) pass@1 by system (HumanEval+)

(b) Average LLM calls per task

Figure 1: Code generation quality and orchestration cost.

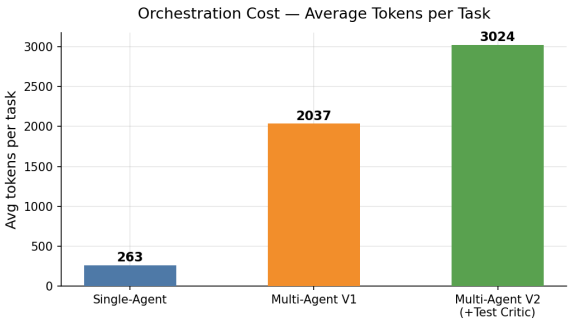
Context-loss bug fix impact (pilot, 10 problems).

5.1.2 Requirements Elicitation

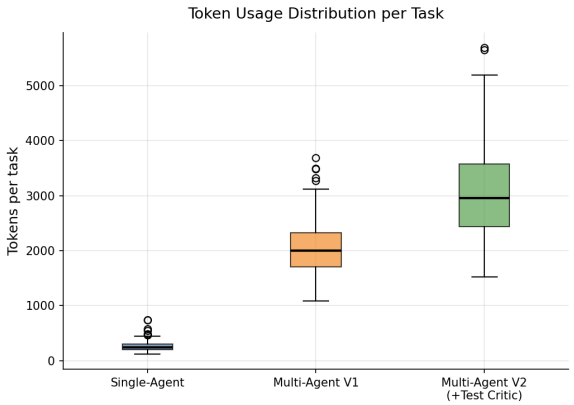
5.1.3 Cross-Task Cost Analysis

5.2 Software Deliverables

The following scripts are provided as runnable deliverables:

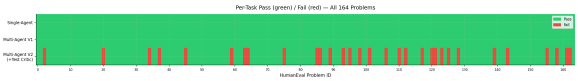


(a) Average tokens per task

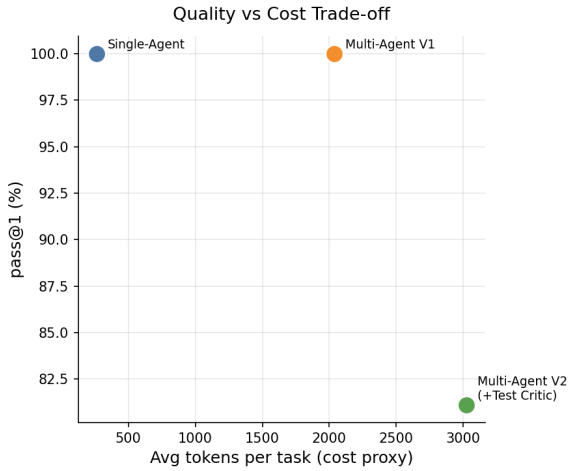


(b) Token distribution (box plot)

Figure 2: Token cost comparison across systems.



(a) Per-task pass/fail heatmap



(b) Quality vs. cost scatter

Figure 3: Per-task analysis and quality/cost trade-off (HumanEval+).

Table 2: V1 BigCodeBench-Hard pilot: before and after context-loss fixes.

Metric	**Before fix**	**After fix**
V1 pilot pass@1 (10 problems)	60%	**90%**
Single-agent pilot pass@1	40%	40%
Repair attempts with plan	0 / 5	5 / 5
Avg. plan tokens in repair	0	417
Unittest tests executed	No	Yes

Table 3: Requirements elicitation results: NICE dataset with BRD-based input. Avg LLM calls: 1.0 (Single-Agent), 3.0 (V1), 4.0 (V2+SME). Token costs in Table 4.

System	**Sem. F1**	**Coverage**	**Precision**	**FR Cov.**	**NFR Cov.**
Single-Agent	**0.781**	**0.852**	**0.730**	**0.905**	**0.771**
Multi-Agent V1	0.689	0.796	0.640	0.905	0.688
Multi-Agent V2+SME	0.638	0.759	0.580	0.837	0.625

Table 4: Tokens per task: Single-Agent vs. V1 across all benchmarks.

Task	Dataset	Single	V1	Overhead	Δ Quality
Code Gen	HumanEval+	263	2,037	7.7×	0
Code Gen	MBPP+	138	1,531	11.1×	0
Code Gen	BigCodeBench-Hard	836	5,718	6.8×	−0.7%
RE	NICE (BRD)	9,537	22,988	2.4×	−11.8%

Δ Quality = ΔF1 (V1 vs. Single-Agent). RE evaluated on NICE dataset with BRD input ($N = 2$ projects).

- `scripts/run_mbpp.py` , Run all 3 systems on MBPP+ with incremental save and resume.
- `scripts/run_bigcodebench.py` , Run all 3 systems on BigCodeBench-Hard with library pre-installation, context trace logging, and resume support.
- `scripts/rerun_bigcode_failures.py` , Re-run only import-error failures in-place, patching existing result files.
- `scripts/run_re_elicitation.py` , Run all 3 systems on NICE and PURE.
- `scripts/evaluate_re_elicitation.py` , Standalone semantic F1 evaluation from saved result files.
- `scripts/prepare_re_elicitation.py` , One-time: generate use-case descriptions from ground-truth requirement lists.

6 Waiting Room

The following items were planned but could not be completed due to time or API credit limitations:

1. **Fixed V1 full run on BigCodeBench-Hard (148 problems).** The post-bugfix run was terminated at 59/148 problems due to API credit exhaustion. The pilot result (90% on 10 problems) is promising but the full run is needed to confirm whether the fixed repair loop consistently outperforms single-agent across the complete benchmark.
2. **Multi-Agent V2 on BigCodeBench-Hard.** V2 was not evaluated on BigCodeBench-Hard. Since the Test Critic only activates on problems that pass, roughly 38% of the dataset, it would have limited scope to influence results. Nevertheless, a full V2 run would complete the comparison.
3. **Manual verification of Test Critic results.** V2 scores ∼18% lower than V1 on HumanEval+ and MBPP+. It is unclear whether this reflects genuine bugs caught by augmented tests or invalid test assertions generated by the critic. A manual review of the 31 HumanEval+ failures and 83 MBPP+ failures was planned but not completed.
4. **Full BigCodeBench-Hard (740 problems).** We evaluated only the v0.1.4 split (148 problems). The complete benchmark spans 5 splits and 740 problems. Evaluating the full set would reduce variance and allow per-category analysis (ML, web, file I/O, systems).
5. **Multi-model comparison.** All results use `claude-sonnet-4-6`. Repeating the study with a second model (e.g. GPT-4o or an open-weight model via Ollama) would test whether the single-agent vs. multi-agent findings generalise across model families.
6. **Harder RE benchmarks.** NICE and PURE both have ceiling effects for single-agent. A dataset with longer, more ambiguous specifications, where single-agent demonstrably fails,

would be needed to determine whether the V1 RE pipeline adds value in the regime where the baseline struggles.

7. **Dynamic MAX_REPAIR_ITERATIONS tuning.** The repair budget was fixed at 3 iterations throughout. An ablation over $\{1, 2, 3, 5\}$ iterations on BigCodeBench-Hard (post-bugfix) would quantify the marginal value of additional repair rounds.

References

[1] Liu, J., et al. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models with CodeEval Plus. *NeurIPS 2023*.

[2] Zhuo, T. Y., et al. (2024). BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv:2406.15877*.

[3] Fantechi, A., et al. (2025). NICE: A Requirements Dataset for Benchmarking Requirements Engineering Tasks. *ICSE 2025*.

[4] Ferrari, A., et al. (2017). PURE: A Dataset of Public Requirements Documents. *IEEE RE 2017*.

[5] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.

[6] LangChain AI. (2024). LangGraph: Building Stateful, Multi-Actor Applications with LLMs. *GitHub*.

[7] Anthropic. (2024). Claude: A Family of Large Language Models. *Anthropic Technical Report*.